claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-ae2c09a47ca97ef01.jsonl`
- SHA-256: `c63726e59ebe0f1aa1777fef0a556d76ac03f87a01ff4b145e04fb2fce9c85b2`
- Source modified: `2026-06-22T09:47:48+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:45:35.962Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

{
 "title": "Re-run guard adopts second-run values without re-validating baseline match ? potential regression mask",
 "file": "scripts/nightly_reelcount.py",
 "line": "362-366",
 "severity": "high",
 "detail": "The re-run guard (lines 362-366) compares count to baseline_count on the first run, and if they mismatch, re-processes the video. If the second run succeeds (err2 is None and count2 is not None), it adopts count2 unconditionally at line 366. However, if count2 != baseline_count, this real change will be reported as a regression in compare(). The guard correctly mitigates transient flakes by re-running, but the logic is fragile: if the re-run produces a DIFFERENT count (baseline?5, run1?3, run2?7), the adoption of count2=7 reports as regression vs baseline. This is actually correct behavior, but the code does NOT re-check whether the second run matched the baseline ? it just adopts it. If baseline=5, run1=3, run2=5 (transient flake), the guard works. But if run1 and run2 both differ from baseline in DIFFERENT ways, the second value masks the first failure mode. This is correct in the current design but relies on the assumption that transient flakes are rare and won't produce two different counts.",
 "fix": "Add a log statement (optional, for observability): before line 366, log when count2 differs from the baseline: `if count2 != baseline_count: log.warning('second run count %d != baseline %d ? regression is real', count2, baseline_count)`. Current behavior is safe but lacks transparency."
}

2. assistant (2026-06-22T09:45:38.566Z)

I'll analyze the nightly reel-count engine-regression PR by examining the key files. Let me start by reading the main script and related files to verify the finding about the re-run guard logic.

3. user (2026-06-22T09:45:40.192Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3       Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4       it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5       vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6       trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7       needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8       owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10      What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11        ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12          Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13          **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see §guard).
14        ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15          ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16        ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18      Extensible pattern (add a video = one manifest row + re-freeze):
19        ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20        ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated,
never hand-edited)
21
22      Usage
23      -----
24      Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25          python scripts/nightly_reelcount.py --update-baseline
26
27      Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28          python scripts/nightly_reelcount.py --email
29
30      Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31      (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32      """
33      from __future__ import annotations
34
35      import argparse
36      import datetime as _dt
37      import json
38      import os
39      import sys
40      from dataclasses import dataclass, field
41      from typing import Any, Dict, List, Optional, Tuple
42
43      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
    if REPO_ROOT not in sys.path:
        sys.path.insert(0, REPO_ROOT)

    # billing is pure (typing only) ? safe to import at module top.
    from backend import billing  # noqa: E402

    DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
    DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
    DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")

    #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
    QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
    DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the
exact reel-count
    DEFAULT_FX_INR_PER_USD = 83.0
    #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
    DEFAULT_MACHINE_USD_PER_HR = 0.35


    @dataclass(frozen=True)
    class Tolerances:
        quality_tol: float = DEFAULT_QUALITY_TOL

        def to_dict(self) -> Dict[str, Any]:
            return {"quality_tol": self.quality_tol}


    # ---------------------------------------------------------------------- #
    # Pure cost math (wraps backend.billing ? no IO).
    # ---------------------------------------------------------------------- #
    def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
        """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).

        ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
        otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
        rate_per_sec = float(machine_usd_per_hr) / 3600.0
        usage = {billing.WALL_SECONDS: float(billed_seconds)}
        rates = {billing.WALL_SECONDS: rate_per_sec}
        usd, breakdown = billing.compute_cost_usd(usage, rates)
        return {
            "billed_seconds": round(float(billed_seconds), 1),
            "gpu_seconds": round(float(gpu_seconds), 1),
            "machine_usd_per_hr": float(machine_usd_per_hr),
            "usd": usd,
            "inr": billing.to_inr(usd, fx_inr_per_usd),
            "breakdown": breakdown,
        }


    # ---------------------------------------------------------------------- #
    # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
    # ---------------------------------------------------------------------- #
    def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
                          cost: Dict[str, Any]) -> Dict[str, Any]:
        """Compact, comparable snapshot from per-video run results.

        Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
        A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
```

```
never hides it)."""
        per_video: Dict[str, Any] = {}
        for r in run_results:
            per_video[r["name"]] = {
                "reel_count": r.get("reel_count"),
                "ok": bool(r.get("ok", False)),
                "error": r.get("error"),
                "quality": r.get("quality"),
            }
        return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
                "n": len(per_video)}


    @dataclass
    class Finding:
        video: str
        metric: str                 # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
        kind: str                   # "regression" | "improvement" | "stale"
        baseline: Optional[float] = None
        current: Optional[float] = None
        detail: str = ""

        def message(self) -> str:
            if self.kind == "stale":
                return f"[STALE] {self.metric}: {self.detail}"
            if self.metric == "error":
                return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
            b = "?" if self.baseline is None else f"{self.baseline:g}"
            c = "?" if self.current is None else f"{self.current:g}"
            tag = "REGRESSION" if self.kind == "regression" else "improvement"
            return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


    @dataclass
    class NightlyResult:
        findings: List[Finding] = field(default_factory=list)
        added: List[str] = field(default_factory=list)
        removed: List[str] = field(default_factory=list)

        @property
        def regressions(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "regression"]

        @property
        def improvements(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "improvement"]

        @property
        def stale(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "stale"]

        def overall_status(self) -> str:
            if self.regressions:
                return "REGRESSION"
            if self.stale:
                return "STALE"
            return "PASS"

        def exit_code(self) -> int:
            if self.regressions:
                return 1
            if self.stale:
                return 4
            return 0
```

```

    def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
        """Diff current run vs frozen baseline.

        * Segmenter changed ? STALE (numbers not comparable; re-freeze).
        * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
        * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
          quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
        """
        res = NightlyResult()
        if baseline.get("segmenter") != current.get("segmenter"):
            res.findings.append(Finding("", "segmenter", "stale",
                                        detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
            return res

        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        res.added = sorted(set(c_pv) - set(b_pv))
        res.removed = sorted(set(b_pv) - set(c_pv))
        if res.added or res.removed:
            res.findings.append(Finding("", "corpus", "stale",
                                        detail=f"added={res.added} removed={res.removed}"))

        for v in sorted(set(b_pv) & set(c_pv)):
            b, c = b_pv[v], c_pv[v]
            # 1) pipeline must still succeed
            if not c.get("ok", False) or c.get("reel_count") is None:
                res.findings.append(Finding(v, "error", "regression",
                                            detail=str(c.get("error") or "no reel_count
produced")))
                continue
            # 2) reel count ? EXACT
            bc, cc = b.get("reel_count"), c.get("reel_count")
            if bc is not None and cc is not None and cc != bc:
                res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
            # 3) quality metrics ? tolerance (only if both sides have them)
            bq, cq = b.get("quality") or {}, c.get("quality") or {}
            for m in QUALITY_HIGHER:
                bv, cv = bq.get(m), cq.get(m)
                if bv is None or cv is None:
                    continue
                if cv < bv - tols.quality_tol:
                    res.findings.append(Finding(v, m, "regression", bv, cv))
                elif cv > bv + tols.quality_tol:
                    res.findings.append(Finding(v, m, "improvement", bv, cv))
        return res


    # ------------------------------------------------------------------------ #
    # Report formatting (pure).
    # ------------------------------------------------------------------------ #
    def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
        if not qd or qd.get(key) is None:
            return "?"
        return f"{qd[key]:.3f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, date: str, head: str) -> str:
        status = result.overall_status()
```

```
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
                 "STALE": "? STALE (re-freeze the baseline)"}[status]
        cost = current.get("cost", {})
        L: List[str] = [
            f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
            "",
            f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
`{current.get('segmenter')}`",
            "",
            f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
            f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
            f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
            "",
        ]
        if result.regressions:
            L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
        if result.stale:
            L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
            L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]

        # Per-video table.
        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        L += ["## Per-video", "",
              "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
              "|---|---|---|---|---|"]
        for v in sorted(set(b_pv) | set(c_pv)):
            b, c = b_pv.get(v, {}), c_pv.get(v, {})
            bc = b.get("reel_count")
            cc = c.get("reel_count")
            rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
            bq, cq = b.get("quality"), c.get("quality")
            tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
            f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
            vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
            L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
        L.append("")

        if result.improvements:
            L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
                + [f"- {f.message()}" for f in result.improvements] + [""]
        L += ["---",
              "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
              "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
        return "\n".join(L)


    # ------------------------------------------------------------------------ #
    # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
    # ------------------------------------------------------------------------ #
    def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
        """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a
rallies CSV
        (``start,end`` columns / pairs). Returns None when no labels are configured."""
        if not labels_ref:
            return None
        from backend.storage import fetch
```

```
        local = fetch(labels_ref)
        if local.endswith(".json"):
            with open(local, encoding="utf-8") as fh:
                data = json.load(fh)
            return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
        # CSV: header start,end (or first two numeric columns)
        import csv
        out: List[Tuple[float, float]] = []
        with open(local, encoding="utf-8") as fh:
            for row in csv.reader(fh):
                try:
                    out.append((float(row[0]), float(row[1])))
                except (ValueError, IndexError):
                    continue
        return out


    def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
        """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
        codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
        optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
        quality scoring gracefully (skipped) without ever miscounting reels."""
        out: List[Tuple[float, float]] = []
        for s in segs:
            start = s.get("start_time", s.get("start"))
            end = s.get("end_time", s.get("end"))
            if start is not None and end is not None:
                out.append((float(start), float(end)))
        return out


    def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                      rerun_on_mismatch: bool = True,
                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
        """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.

        The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
        database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
        reproduce, a real change will."""
        import random
        import time

        from backend.eval import rally_seg_eval
        from backend.infrastructure.database import Database
        from backend.pipeline.segmenters import segmenter_registry
        from backend.results import Failure
        from backend.storage import fetch
        from backend.utils.hashing import compute_video_id

        name = video["name"]
        try:
            local = fetch(video["uri"])
        except Exception as e:  # noqa: BLE001 ? surface, never hide
            return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
                    "quality": None, "wall_seconds": 0.0}

        gts = _load_gts(video.get("labels_uri"))
        seg_cls = segmenter_registry.get(segmenter)
```

```
        if not seg_cls:
            return {"name": name, "reel_count": None, "ok": False,
                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}

        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
        with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
            count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
            total_wall += wall
            if err is not None:
                return {"name": name, "reel_count": None, "ok": False, "error": err,
                        "quality": None, "wall_seconds": round(total_wall, 2)}
            # re-run-once guard for the transient DB-drop flake
            if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
                count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected)
value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}


    def load_manifest(path: str) -> Dict[str, Any]:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
        """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
        from backend.config import load_config

        config = load_config()
        # apply manifest config overrides onto the typed config (dotted keys)
        for dotted, val in (manifest.get("config_overrides") or {}).items():
            _set_dotted(config, dotted, val)
        segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
        b_pv = (baseline or {}).get("per_video", {})
```

```
        results: List[Dict[str, Any]] = []
        total_wall = 0.0
        for v in manifest.get("videos", []):
            base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
            r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
            total_wall += float(r.get("wall_seconds") or 0.0)
            results.append(r)
        return results, total_wall


    def _set_dotted(config: Any, dotted: str, value: Any) -> None:
        """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
        parts = dotted.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p, None)
            if obj is None:
                return
        try:
            setattr(obj, parts[-1], value)
        except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
            pass


    def _git_head() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:  # noqa: BLE001
            return "?"


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_head()
        out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
                       "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
                       "configured+checkpointed engine + the manifest corpus.")
        out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST)
```

```
        p.add_argument("--baseline", default=DEFAULT_BASELINE)
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
        p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
        p.add_argument("--vm-uptime-seconds", type=float, default=None,
                       help="authoritative VM uptime (boot?delete) for the cost line; "
                            "defaults to the in-process pipeline wall time (a lower bound)")
        p.add_argument("--no-rerun-guard", action="store_true",
                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
        p.add_argument("--update-baseline", action="store_true",
                       help="freeze the current reel counts + quality as the new baseline
instead of checking")
        p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
        p.add_argument("--email-to", default=None, help="override the report recipient")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_reelcount")
        args = build_parser().parse_args(argv)
        tols = Tolerances(quality_tol=args.quality_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s", args.manifest)
            return 2
        manifest = load_manifest(args.manifest)
        baseline = load_baseline(args.baseline)

        try:
            run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
        except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
            log.error("run failed: %s", e)
            return 2
        if not run_results:
            log.error("no videos in manifest %s", args.manifest)
            return 2

        billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
        cost = cost_report(
            billed_seconds=billed,
            machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
            fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
            gpu_seconds=total_wall,
        )
        segmenter = manifest.get("segmenter", "motion")
        snapshot = summarize_results(run_results, segmenter, cost)
        snapshot["git_commit"] = _git_head()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot)
            log.info("baseline frozen ? %s", args.baseline)
            for r in run_results:
```

```
510                 log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511             return 0
512
513         if baseline is None:
514             log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515             return 3
516
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                            "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554
```

4. user (2026-06-22T09:45:40.258Z)

```
1       """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3       Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4       dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
```

```
password can
5       be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6       never mask a regression).
7
8       Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9         ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
10        ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11        ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
12        ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
13                      (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14                       lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
the simplest.
15
16      Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17      and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18                      export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19      """
20      from __future__ import annotations
21
22      import logging
23      import os
24      from email.message import EmailMessage
25      from typing import Optional
26
27      log = logging.getLogger("nightly_email")
28
29      DEFAULT_TO = "avi.dullu@gmail.com"
30      DEFAULT_HOST = "smtp.gmail.com"
31      DEFAULT_PORT = 587
32
33
34      def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35          """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36          msg = EmailMessage()
37          msg["Subject"] = subject
38          msg["From"] = from_addr
39          msg["To"] = to_addr
40          msg.set_content(body_md)
41          return msg
42
43
44      def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45          """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46          env = env if env is not None else os.environ
47          direct = env.get("NIGHTLY_SMTP_PASS")
48          if direct:
49              return direct
50          secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51          if not secret_ref:
52              return None
53          # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54          try:
55              from google.cloud import secretmanager  # type: ignore
56              client = secretmanager.SecretManagerServiceClient()
57              return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
```

```
        except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
            pass
        try:
            import subprocess
            name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
            out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
                                 capture_output=True, text=True, timeout=30)
            return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
        except Exception as e:  # noqa: BLE001
            log.warning("secret-manager resolution failed: %s", e)
            return None


    def send_report(subject: str, body_md: str, to: Optional[str] = None,
                    env: Optional[dict] = None) -> bool:
        """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
        env = env if env is not None else os.environ
        to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
        host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
        port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
        user = env.get("NIGHTLY_SMTP_USER") or to_addr
        password = resolve_secret(env)
        if not password:
            log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
            return False

        import smtplib
        msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
        with smtplib.SMTP(host, port, timeout=60) as s:
            s.starttls()
            s.login(user, password)
            s.send_message(msg)
        log.info("emailed report to %s", to_addr)
        return True

```

5. user (2026-06-22T09:45:40.990Z)

```
      {
        "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
        "segmenter": "wasb_hybrid",
        "detector_impl": "native",
        "config_overrides": {
          "indexing.default_segmenter": "wasb_hybrid",
          "indexing.detector_impl": "native"
        },
        "machine_usd_per_hr": 0.70,
        "fx_inr_per_usd": 83.0,
        "videos": [
          {
            "name": "GX030094",
            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
            "labels_uri": "gs://khelsutra-rally-
```

```
corpus/nightly/labels/GX030094_golden_features.json"
16          },
17          {
18            "name": "Badminton_BXH_2",
19            "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21          },
22          {
23            "name": "GX010128",
24            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26          }
27        ]
28      }
29
```

6. assistant (2026-06-22T09:45:43.426Z)

Now let me look at the specific lines mentioned in the finding (362-366) and the surrounding context more carefully:

7. user (2026-06-22T09:45:45.021Z)

```
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
360                         "quality": None, "wall_seconds": round(total_wall, 2)}
361             # re-run-once guard for the transient DB-drop flake
362             if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363                 count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364                 total_wall += wall2
365                 if err2 is None and count2 is not None:
366                     count, intervals = count2, intervals2  # the reproduced (or corrected)
value
367
368             quality = None
369             if gts and intervals is not None:
370                 row = rally_seg_eval.score_intervals(intervals, gts)
371                 quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
372                            if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                 "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
```

8. assistant (2026-06-22T09:45:46.987Z)

Now let me check the deploy files and understand the full workflow:

9. user (2026-06-22T09:45:47.731Z)

```
EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\deploy\gcp\nightly_regression'
```

10. user (2026-06-22T09:45:50.790Z)

deploy\gcp\nightly_regression\run_on_vm.sh
deploy\gcp\nightly_regression\launch.sh
deploy\gcp\nightly_regression\Dockerfile.launcher
deploy\gcp\nightly_regression\register_scheduler.sh
deploy\gcp\nightly_regression\README.md

11. user (2026-06-22T09:45:51.577Z)

```
1       #!/usr/bin/env bash
2       # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
3       # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
4       # an interactive session). Reads its config from instance metadata (set by launch.sh).
5       #
6       # Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage
weights  ?
7       #        run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8       #        upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
9       #
10      # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11      #   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12      set -uo pipefail
13      exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14      echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
15      BOOT_EPOCH=$(date +%s)
16
17      md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18      NAME=$(md instance/name)
19      ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20      REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21      GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22      BUCKET=$(md instance/attributes/gcs-bucket || true)
23      REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24      WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25      DO_EMAIL=$(md instance/attributes/email || echo true)
26      SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27      SMTP_USER=$(md instance/attributes/smtp-user || true)
28      EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29      DATE=$(date -u +%F)
30
31      # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32      cleanup() {
33        echo "===== self-delete $NAME ($ZONE) ====="
34        gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35          echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36      }
37      trap cleanup EXIT
38
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
```

```
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49      PY
50        fi
51      }
52      trap 'fail_email' ERR
53
54      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59      mkdir -p ~/rally && cd ~/rally
60      gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61      tar -xzf /tmp/rally.tgz -C ~/rally
62      echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64      # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65      sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66      export TMPDIR=/scratch 2>/dev/null || true
67      ./deploy/digitalocean/bootstrap.sh --gpu
68      . .venv/bin/activate
69      pip install -e '.[storage-gcs]'
70      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73      if [ -n "$WEIGHTS_URI" ]; then
74        mkdir -p ~/models/WASB-SBDT/pretrained_weights
75        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76        export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77      fi
78
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91        gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
94
95      echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96      trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
97      # the EXIT trap deletes the VM
98
```

12. assistant (2026-06-22T09:45:54.256Z)

Now let me check the compare function more carefully and understand what happens when the second run produces a different count:

13. user (2026-06-22T09:45:55.302Z)

```
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173           quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                         detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
179             return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))
184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",
186                                         detail=f"added={res.added} removed={res.removed}"))
187
188         for v in sorted(set(b_pv) & set(c_pv)):
189             b, c = b_pv[v], c_pv[v]
190             # 1) pipeline must still succeed
191             if not c.get("ok", False) or c.get("reel_count") is None:
192                 res.findings.append(Finding(v, "error", "regression",
193                                             detail=str(c.get("error") or "no reel_count
produced")))
194                 continue
195             # 2) reel count ? EXACT
196             bc, cc = b.get("reel_count"), c.get("reel_count")
197             if bc is not None and cc is not None and cc != bc:
198                 res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199             # 3) quality metrics ? tolerance (only if both sides have them)
200             bq, cq = b.get("quality") or {}, c.get("quality") or {}
201             for m in QUALITY_HIGHER:
202                 bv, cv = bq.get(m), cq.get(m)
203                 if bv is None or cv is None:
204                     continue
205                 if cv < bv - tols.quality_tol:
206                     res.findings.append(Finding(v, m, "regression", bv, cv))
207                 elif cv > bv + tols.quality_tol:
208                     res.findings.append(Finding(v, m, "improvement", bv, cv))
209         return res
210
211
```

14. assistant (2026-06-22T09:45:57.680Z)

Now let me trace the full logic flow and understand the issue. Let me look at the actual calling of run_one_video:

15. user (2026-06-22T09:45:58.584Z)

```
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                    rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387 
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402 
```

16. assistant (2026-06-22T09:46:14.111Z)

Perfect. Now I have all the context I need. Let me analyze the logic carefully:

The re-run guard logic (lines 362-366):

1. **Line 362**: `if rerun_on_mismatch and baseline_count is not None and count != baseline_count:` - Triggers re-run when first run count doesn't match baseline
2. **Line 363-366**: If the second run succeeds (no error), it unconditionally adopts `count2` and `intervals2`

The comparison logic (lines 196-198 in `compare()`):
- Compares current's `reel_count` vs baseline's `reel_count`
- ANY difference = regression

Scenario analysis:
- If `baseline_count=5, count1=3, count2=7`:
  - First run: 3 != 5, so re-run triggers
  - Second run: count2=7 succeeds
  - Line 366 adopts count2=7
  - Compare: 5 != 7 ? REGRESSION reported ? (correct)

- If `baseline_count=5, count1=3, count2=5`:
  - First run: 3 != 5, so re-run triggers
  - Second run: count2=5 succeeds
  - Line 366 adopts count2=5
  - Compare: 5 == 5 ? PASS ? (correct - transient flake mitigated)

- If `baseline_count=5, count1=3, count2=4`:
  - First run: 3 != 5, so re-run triggers
  - Second run: count2=4 succeeds
  - Line 366 adopts count2=4
  - Compare: 5 != 4 ? REGRESSION reported ? (correct)

The finding claims the code lacks "transparency" because it doesn't re-check whether the second run matched the baseline before adopting it. Let me verify whether this is actually a problem:

The docstring at line 314 says: "The re-run-once guard (the one known non-determinism ? a transient ``add_play_segment``?None drop, database.py): if the count != the frozen baseline, re-process the video ONCE; a transient drop won't reproduce, a real change will."

This reveals the **design intent**: the re-run is meant to handle transient flakes where run1

mistakenly drops segments. If run2 produces a different count than baseline, that's a REAL change (regression), and it should be reported as such. The code correctly reports this via the `compare()` function.

Key insight: The finding is incorrect about this being a defect. The logic is sound:
1. If count1 != baseline, re-run once
2. If count2 succeeds, use it (whether or not it matches baseline)
3. compare() will report any count2 != baseline as regression

This is actually the CORRECT behavior because:
- Transient flakes won't reproduce (count2 will equal baseline)
- Real regressions WILL show in count2, and compare() WILL catch them
- There's no logic that could mask a real regression

Let me verify there are no edge cases in error handling:

17. user (2026-06-22T09:46:15.069Z)

```
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
360                         "quality": None, "wall_seconds": round(total_wall, 2)}
361             # re-run-once guard for the transient DB-drop flake
362             if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363                 count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364                 total_wall += wall2
365                 if err2 is None and count2 is not None:
366                     count, intervals = count2, intervals2  # the reproduced (or corrected)
value
367
```

18. assistant (2026-06-22T09:46:21.704Z)

Now I need to think about a subtle issue: **what if err2 is None but count2 is None?** Looking at line 340-351, the `_process()` function returns `(len(segs), intervals, wall, None)` on success. The only way to get `err2 is None` but `count2 is None` would be if `len(segs)` somehow returned None, which is impossible since `len()` always returns an int.

However, there's another subtle case: **what if the second run also fails (err2 is not None)?** In that case, line 365's condition is false, so the code does NOT adopt count2. Instead, it keeps the original count1 from line 356. This means:
- If count1 != baseline_count and the re-run fails, count1 (which is != baseline) is retained
- compare() will then report count1 vs baseline as a regression

But this could be MISLEADING if count1 was a transient flake and the second run crashed for an unrelated reason. The user would see a "regression" when it was actually just a transient DB

flake followed by an unrelated crash.

Let me check if this is the real defect:

19. user (2026-06-22T09:46:22.759Z)

```
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
360                         "quality": None, "wall_seconds": round(total_wall, 2)}
361             # re-run-once guard for the transient DB-drop flake
362             if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363                 count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364                 total_wall += wall2
365                 if err2 is None and count2 is not None:
366                     count, intervals = count2, intervals2  # the reproduced (or corrected)
value
367
368             quality = None
369             if gts and intervals is not None:
370                 row = rally_seg_eval.score_intervals(intervals, gts)
371                 quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
372                            if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                 "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
```

20. assistant (2026-06-22T09:46:28.656Z)

Now I see the real issue:

If `count1 != baseline_count` and `err2 is not None` (second run fails):
- Line 365's condition is false
- count is **NOT updated** from line 366
- count remains = count1 (which was != baseline_count)
- Line 374 returns `{"ok": True, ...}` with count = count1
- compare() sees count1 != baseline_count and reports REGRESSION

But this is WRONG because:
1. The first run had a transient flake (count1 != baseline)
2. The second run crashed for some unrelated reason
3. We're now reporting the transient flake as a "regression" instead of an "error"

The function should either:
- Return `ok=False` if the second run fails AND the first run had a mismatch
- Or skip the re-run guard and just return ok=True with count1 if the second run fails
- Or log a warning that the re-run validation was inconclusive

Let me check if there are any other callers or tests that shed light on the intended behavior:

21. user (2026-06-22T09:46:29.422Z)

Found 1 file
scripts\nightly_reelcount.py

22. user (2026-06-22T09:46:32.536Z)

(Bash completed with no output)

23. user (2026-06-22T09:46:35.284Z)

```
./tests/test_nightly_reelcount.py
./tests/test_nightly_regression.py
./tests/__pycache__/test_nightly_reelcount.cpython-313-pytest-9.0.3.pyc
./tests/__pycache__/test_nightly_regression.cpython-313-pytest-9.0.3.pyc
```

24. user (2026-06-22T09:46:37.687Z)

```
1       """Tests for scripts/nightly_reelcount.py + scripts/nightly_email.py ? the pure cores.
2
3       Fully offline: no GPU, no video, no network, no real corpus. The comparison / cost /
report / email
4       formatting operate on plain dicts + strings, so reel-count regressions, quality drops,
pipeline
5       errors, corpus drift, and the no-creds email skip are all exercised deterministically.
The actual
6       pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on
the VM.
7       """
8       import importlib.util
9       import os
10      import sys
11
12      import pytest
13
14      _SCRIPTS = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
"scripts")
15
16
17      def _load(modname):
18          spec = importlib.util.spec_from_file_location(modname, os.path.join(_SCRIPTS,
f"{modname}.py"))
19          mod = importlib.util.module_from_spec(spec)
20          sys.modules[modname] = mod  # register before exec so dataclass annotations resolve
21          spec.loader.exec_module(mod)
22          return mod
23
24
25      nr = _load("nightly_reelcount")
26      ne = _load("nightly_email")
27
28
29      # ---------------------------------------------------------------------------- #
30      # Builders
31      # ---------------------------------------------------------------------------- #
32      def _video(name, reel_count, ok=True, error=None, quality=None):
33          return {"name": name, "reel_count": reel_count, "ok": ok, "error": error,
34                  "quality": quality, "wall_seconds": 1.0}
35
36
37      def _snap(per_video, segmenter="wasb_hybrid"):
38          return {"segmenter": segmenter, "per_video": per_video,
39                  "cost": {"usd": 0.05, "inr": 4.15, "billed_seconds": 500,
"machine_usd_per_hr": 0.35}}
40
41
42      TOLS = nr.Tolerances()
43
44
45      # ---------------------------------------------------------------------------- #
46      # cost_report (billing wrapper)
47      # ---------------------------------------------------------------------------- #
```

```
    def test_cost_report_math():
        c = nr.cost_report(billed_seconds=3600, machine_usd_per_hr=0.35,
fx_inr_per_usd=83.0)
        assert c["usd"] == pytest.approx(0.35, abs=1e-4)        # 1hr @ $0.35
        assert c["inr"] == pytest.approx(0.35 * 83.0, abs=0.01)
        assert c["billed_seconds"] == 3600


    def test_cost_report_zero_for_zero_time():
        c = nr.cost_report(billed_seconds=0, machine_usd_per_hr=0.35, fx_inr_per_usd=83.0)
        assert c["usd"] == 0.0 and c["inr"] == 0.0


    # ---------------------------------------------------------------------- #
    # summarize_results
    # ---------------------------------------------------------------------- #
    def test_summarize_results():
        rr = [_video("v1", 5), _video("v2", 3, quality={"tol_f1": 0.4})]
        snap = nr.summarize_results(rr, "wasb_hybrid", {"usd": 0.1})
        assert snap["n"] == 2
        assert snap["per_video"]["v1"]["reel_count"] == 5
        assert snap["per_video"]["v2"]["quality"]["tol_f1"] == 0.4


    # ---------------------------------------------------------------------- #
    # compare ? reel count is EXACT
    # ---------------------------------------------------------------------- #
    def test_identical_counts_pass():
        base = _snap({"v1": {"reel_count": 5, "ok": True}})
        cur = _snap({"v1": {"reel_count": 5, "ok": True}})
        res = nr.compare(base, cur, TOLS)
        assert res.overall_status() == "PASS" and res.exit_code() == 0


    def test_reel_count_change_is_regression():
        base = _snap({"v1": {"reel_count": 5, "ok": True}})
        cur = _snap({"v1": {"reel_count": 4, "ok": True}})       # one reel vanished
        res = nr.compare(base, cur, TOLS)
        assert res.exit_code() == 1
        r = res.regressions[0]
        assert r.metric == "reel_count" and r.baseline == 5 and r.current == 4


    def test_reel_count_increase_is_also_regression():
        # ANY change to the count is a regression ? the count must be stable, either
direction.
        base = _snap({"v1": {"reel_count": 5, "ok": True}})
        cur = _snap({"v1": {"reel_count": 6, "ok": True}})
        assert nr.compare(base, cur, TOLS).exit_code() == 1


    def test_pipeline_error_is_regression():
        base = _snap({"v1": {"reel_count": 5, "ok": True}})
        cur = _snap({"v1": {"reel_count": None, "ok": False, "error": "WASB OOM"}})
        res = nr.compare(base, cur, TOLS)
        assert res.exit_code() == 1
        assert res.regressions[0].metric == "error" and "WASB OOM" in
res.regressions[0].detail


    # ---------------------------------------------------------------------- #
    # compare ? quality metrics use tolerance
    # ---------------------------------------------------------------------- #
    def test_quality_drop_beyond_tol_is_regression():
        base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
```

```
        cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.30}}})   # ?0.10 ? tol
        res = nr.compare(base, cur, TOLS)
        assert res.exit_code() == 1 and res.regressions[0].metric == "tol_f1"


    def test_quality_drop_within_tol_passes():
        base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
        cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.39}}})   # within 0.02
        assert nr.compare(base, cur, TOLS).exit_code() == 0


    def test_quality_rise_is_improvement_not_regression():
        base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.40}}})
        cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.55}}})
        res = nr.compare(base, cur, TOLS)
        assert res.exit_code() == 0
        assert [f.metric for f in res.improvements] == ["f1@0.5"]


    # ------------------------------------------------------------------------ #
    # compare ? staleness (segmenter / corpus drift)
    # ------------------------------------------------------------------------ #
    def test_segmenter_change_is_stale():
        base = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="wasb_hybrid")
        cur = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="motion")
        res = nr.compare(base, cur, TOLS)
        assert res.overall_status() == "STALE" and res.exit_code() == 4


    def test_corpus_drift_is_stale_but_checks_intersection():
        base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok": True}})
        cur = _snap({"v1": {"reel_count": 9, "ok": True}, "v3": {"reel_count": 1, "ok": True}})
        res = nr.compare(base, cur, TOLS)
        assert "v3" in res.added and "v2" in res.removed
        # v1 regressed on the intersection ? regression wins over stale (exit 1)
        assert res.exit_code() == 1
        assert any(f.video == "v1" and f.metric == "reel_count" for f in res.regressions)


    def test_corpus_drift_clean_intersection_is_stale_exit_4():
        base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok": True}})
        cur = _snap({"v1": {"reel_count": 5, "ok": True}})       # v2 dropped, v1 unchanged
        res = nr.compare(base, cur, TOLS)
        assert res.exit_code() == 4 and res.overall_status() == "STALE"


    # ------------------------------------------------------------------------ #
    # report formatting (smoke)
    # ------------------------------------------------------------------------ #
    def test_format_report_has_status_cost_and_regression():
        base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.4, "f1@0.5": 0.5}}})
        base["git_commit"] = "aaa"
        base["generated_at"] = "2026-06-21"
        cur = _snap({"v1": {"reel_count": 3, "ok": True, "quality": {"tol_f1": 0.4, "f1@0.5": 0.5}}})
        res = nr.compare(base, cur, TOLS)
        rpt = nr.format_report(base, cur, res, "2026-06-22", "bbb")
        assert "Nightly engine regression" in rpt
        assert "REGRESSION" in rpt
```

```
168         assert "Run cost:" in rpt and "?" in rpt
169         assert "reel_count" in rpt
170         assert "v1" in rpt
171
172
173     def test_format_report_pass_badge():
174         base = _snap({"v1": {"reel_count": 5, "ok": True}})
175         cur = _snap({"v1": {"reel_count": 5, "ok": True}})
176         res = nr.compare(base, cur, TOLS)
177         assert "PASS" in nr.format_report(base, cur, res, "2026-06-22", "bbb")
178
179
180     # ---------------------------------------------------------------------- #
181     # email ? build + creds resolution + graceful skip
182     # ---------------------------------------------------------------------- #
183     def test_email_build_message():
184         msg = ne.build_message("subj", "body **md**", "from@x.com", "to@y.com")
185         assert msg["Subject"] == "subj" and msg["To"] == "to@y.com" and msg["From"] ==
"from@x.com"
186         assert "body **md**" in msg.get_content()
187
188
189     def test_email_resolve_secret_from_env():
190         assert ne.resolve_secret({"NIGHTLY_SMTP_PASS": "hunter2"}) == "hunter2"
191
192
193     def test_email_resolve_secret_none_when_absent():
194         assert ne.resolve_secret({}) is None
195
196
197     def test_email_send_skips_without_creds():
198         # No password resolvable ? graceful skip (False), never raises, no network.
199         assert ne.send_report("s", "b", to="x@y.com", env={"NIGHTLY_EMAIL_TO": "x@y.com"})
is False
200
201
202     def test_tolerances_default():
203         assert nr.Tolerances().quality_tol == 0.02
204
205
206     def test_segments_to_intervals_both_key_conventions():
207         # motion/DB shape (start_time/end_time) AND the alternate (start/end) both parse;
bad rows skip.
208         segs = [{"start_time": 1.0, "end_time": 2.0}, {"start": 5.0, "end": 7.0}, {"nope":
1}]
209         assert nr._segments_to_intervals(segs) == [(1.0, 2.0), (5.0, 7.0)]
210         assert nr._segments_to_intervals([]) == []
211
```

25. assistant (2026-06-22T09:46:49.719Z)

The tests don't cover the re-run guard logic (which is noted in the comment at line 6: "The actual pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on the VM").

Now let me think critically about the finding again. The issue statement says:

> "If count2 != baseline_count, this real change will be reported as a regression in compare(). The guard correctly mitigates transient flakes by re-running, but the logic is fragile: if the re-run produces a DIFFERENT count (baseline?5, run1?3, run2?7), the adoption of count2=7 reports as regression vs baseline. This is actually correct behavior, but the code does NOT re-check whether the second run matched the baseline ? it just adopts it."

The finding is saying the behavior is correct but claims there's a transparency issue. However, looking more carefully, I believe there's a **REAL DEFECT** that the finding misses:

The actual bug: When the second run fails (err2 is not None), the first run's mismatched count is silently reported as ok=True

Looking at lines 362-366:
```python
if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
    count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
    total_wall += wall2
    if err2 is None and count2 is not None:
        count, intervals = count2, intervals2
```

If err2 is not None:
- The condition on line 365 is false
- `count` is NOT updated; it remains the mismatched count1
- The function continues to line 374 and returns `{"ok": True, ...}` with count=count1 (which != baseline)
- This gets reported as a regression, but it should actually be an ERROR since the re-run validation failed

Let me verify this is a real issue by checking if there's any other handling:

26. user (2026-06-22T09:46:50.994Z)

```
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                       rerun_on_mismatch: bool = True,
311                       baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient ``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e:  # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed: {e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None, "wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]]:
341             random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are reproducible too
342             db = Database(db_path)
```

```
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
360                         "quality": None, "wall_seconds": round(total_wall, 2)}
361             # re-run-once guard for the transient DB-drop flake
362             if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363                 count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364                 total_wall += wall2
365                 if err2 is None and count2 is not None:
366                     count, intervals = count2, intervals2  # the reproduced (or corrected)
value
367
368             quality = None
369             if gts and intervals is not None:
370                 row = rally_seg_eval.score_intervals(intervals, gts)
371                 quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
372                            if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                 "quality": quality, "wall_seconds": round(total_wall, 2)}
376
```

27. assistant (2026-06-22T09:47:01.322Z)

Now I've confirmed the issue. Let me trace through the specific problematic scenario:

Scenario: Transient flake on run1, but run2 also fails (err2 != None)
- baseline_count = 5
- run1: count = 3 (transient flake), err = None
- Line 358-360: err is None, so continue (no early return)
- Line 362: rerun_on_mismatch && baseline_count is not None && count != baseline_count ? TRUE (3 != 5)
- Line 363-364: Run again, get count2 = None, err2 = "WASB OOM"
- Line 365: err2 is None && count2 is not None ? FALSE (err2 is not None)
- count is NOT updated; it stays = 3
- Line 374: Returns `{"ok": True, "reel_count": 3, ...}`
- In compare(): baseline=5, current=3 ? REGRESSION reported

But the reality is:
- Run 1 had a transient DB flake (count=3)
- Run 2 crashed due to WASB OOM (unrelated error)
- This should be reported as an ERROR (validation inconclusive), NOT a regression

The fix would be to return `ok=False` if the re-run was attempted but failed:

```python
if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
    count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
    total_wall += wall2
    if err2 is None and count2 is not None:
```

```
        count, intervals = count2, intervals2
    else:
        # Re-run was attempted but failed; validation inconclusive
        return {"name": name, "reel_count": None, "ok": False,
                "error": f"transient mismatch (run1={count}) + rerun failed: {err2}",
                "quality": None, "wall_seconds": round(total_wall, 2)}
```

OR log a warning that the validation is inconclusive. Now let me verify this is actually a production issue by checking if there are any protections higher up:

28. user (2026-06-22T09:47:02.871Z)

```
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478 
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484 
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487         except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488             log.error("run failed: %s", e)
489             return 2
490         if not run_results:
491             log.error("no videos in manifest %s", args.manifest)
492             return 2
493 
494         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495         cost = cost_report(
496             billed_seconds=billed,
497             machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498             fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499             gpu_seconds=total_wall,
500         )
501         segmenter = manifest.get("segmenter", "motion")
502         snapshot = summarize_results(run_results, segmenter, cost)
503         snapshot["git_commit"] = _git_head()
504         snapshot["generated_at"] = date
505 
506         if args.update_baseline:
507             save_baseline(args.baseline, snapshot)
508             log.info("baseline frozen ? %s", args.baseline)
509             for r in run_results:
510                 log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511             return 0
512 
513         if baseline is None:
514             log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515             return 3
516 
```

```
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                            "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
```

29. assistant (2026-06-22T09:47:05.163Z)

Looking at the compare function again and the summarize_results:

30. user (2026-06-22T09:47:06.754Z)

```
96      def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                            cost: Dict[str, Any]) -> Dict[str, Any]:
98          """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
```

```
109             }
110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
111                 "n": len(per_video)}
```

31. assistant (2026-06-22T09:47:08.377Z)

And looking at compare again:

32. user (2026-06-22T09:47:09.485Z)

```
188         for v in sorted(set(b_pv) & set(c_pv)):
189             b, c = b_pv[v], c_pv[v]
190             # 1) pipeline must still succeed
191             if not c.get("ok", False) or c.get("reel_count") is None:
192                 res.findings.append(Finding(v, "error", "regression",
193                                             detail=str(c.get("error") or "no reel_count
produced")))
194                 continue
195             # 2) reel count ? EXACT
196             bc, cc = b.get("reel_count"), c.get("reel_count")
197             if bc is not None and cc is not None and cc != bc:
198                 res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199             # 3) quality metrics ? tolerance (only if both sides have them)
200             bq, cq = b.get("quality") or {}, c.get("quality") or {}
201             for m in QUALITY_HIGHER:
202                 bv, cv = bq.get(m), cq.get(m)
203                 if bv is None or cv is None:
204                     continue
205                 if cv < bv - tols.quality_tol:
206                     res.findings.append(Finding(v, m, "regression", bv, cv))
207                 elif cv > bv + tols.quality_tol:
208                     res.findings.append(Finding(v, m, "improvement", bv, cv))
209         return res
```

33. assistant (2026-06-22T09:47:17.854Z)

Perfect. Now I have a complete understanding. The critical issue is:

When the second run fails (err2 is not None), the code returns `ok=True` with a mismatched count, which then gets flagged as a "regression" instead of an "error".

However, looking at line 191 in the compare function:
```python
if not c.get("ok", False) or c.get("reel_count") is None:
```

This checks if `ok=False` OR `reel_count is None`. If the code returns `ok=True, reel_count=3`, this check does NOT trigger (because ok=True). So the code would then proceed to check reel_count (line 197) and report it as a regression.

BUT if the code returned `ok=False, reel_count=None`, the compare function WOULD catch it at line 191-194 and report it as an "error" (regression of type "error").

This IS a real defect. The scenario is:
1. Run1 produces count=3 (baseline=5, mismatch triggers re-run)
2. Run2 fails with OOM or timeout
3. Current behavior: returns ok=True, reel_count=3 ? compare() reports REGRESSION (reel_count changed)
4. Correct behavior: should return ok=False, reel_count=None ? compare() reports ERROR (validation failed)

The difference is critical in production because:
- A "regression" alert suggests the engine quality degraded (the model changed)

- An "error" alert suggests an operational issue (re-run validation inconclusive)

Now, let me verify my understanding is correct by creating a test scenario that demonstrates this:

34. user (2026-06-22T09:47:22.710Z)

```
Mismatch detected: 3 != 5, running re-run guard
Re-run failed (err2=WASB OOM, count2=None), keeping count=3
Final result: count=3, ok=True

This will be reported as a REGRESSION (count=3 != baseline=5)
But it should be reported as an ERROR (re-run validation inconclusive)
```

35. assistant (2026-06-22T09:47:29.471Z)

Perfect. This confirms the defect. Now let me check if the finding statement itself was accurate about what was claimed:

Looking back at the finding provided:
> "If count2 != baseline_count, this real change will be reported as a regression in compare(). The guard correctly mitigates transient flakes by re-running, but the logic is fragile: if the re-run produces a DIFFERENT count (baseline?5, run1?3, run2?7), the adoption of count2=7 reports as regression vs baseline. This is actually correct behavior, but the code does NOT re-check whether the second run matched the baseline ? it just adopts it."

This description is **partially correct but misses the real defect**. The finding focuses on the scenario where the second run succeeds but produces a different value (run1=3, run2=7). In that case, it's correctly reported as a regression.

The **actual defect** is when the second run **fails** (err2 != None). In that case:
- The code keeps count1 (which is != baseline)
- Returns ok=True
- Gets reported as a regression instead of an error

Let me create one final verification to ensure this is production-impacting:

36. user (2026-06-22T09:47:30.898Z)

```
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91        gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
```

37. assistant (2026-06-22T09:47:41.028Z)

Yes, this runs in production on a GCP VM. The defect could cause false regression alarms (when really it's an operational issue like WASB OOM during re-run).

Now let me finalize my analysis:

Summary of Findings

The finding provided claims there's a "transparency" issue where the code doesn't re-check if the second run matched the baseline. While the specific scenario described (run1 ? baseline, run2 ? baseline ? regression reported) is actually CORRECT behavior, there **IS a real defect** that the finding misses:

REAL DEFECT: When the re-run fails (err2 != None), the code incorrectly reports a transient mismatch as a regression instead of an error.

Scenario:
- baseline=5, run1=3 (transient flake), run2 crashes (err2="WASB OOM")
- Current code: returns ok=True, reel_count=3 ? reported as REGRESSION
- Correct code: should return ok=False, reel_count=None ? reported as ERROR

This is production-impacting because false regression alarms could mask operational issues.

However, the finding as stated is NOT entirely inaccurate?it just focuses on a different (but correct) scenario. Let me call StructuredOutput with my determination:

38. user (2026-06-22T09:47:48.027Z)

Structured output provided successfully